

Politecnico di Torino

Department of Control and Computer Engineering (DAUIN)

LARGE LANGUAGE MODELS FOR SOFTWARE ENGINEERING

Official Examination Session – January 29, 2025

Date: January 29, 2025 | **Time:** 17:11 – 18:26 (1h 15m) | **Total Questions:** 12 | **Max Score:** 18.00 Points

Contents

1	Part I: Exam Questions	2
2	Part II: Comprehensive Solutions & In-Depth Analysis	7

1 Part I: Exam Questions

Question 1

[1.0 Point | -25% Penalty]

Which of the following are benefits of a Retrieval-Augmented-Generation (RAG) approach to prompting?

- (a) It is possible to adopt self-critique mechanisms to refine the outputs of the agent.
- (b) It is possible to reduce the amount of false information provided to the final user.
- (c) It is possible to prevent the obsolescence of answers by using up-to-date documents.
- (d) It is possible to reduce the amount of hallucinations by providing a fixed structure for the agent's answers.
- (e) It is possible to decompose a task in several smaller steps.

Question 2

[1.0 Point | -25% Penalty]

In-context Learning has been a fundamental paradigm shift for Language Models because:

- (a) It replaces the need for attention mechanisms by introducing a sequential input processing.
- (b) It improves the efficiency of training by compressing longer sequences into fixed-size representations.
- (c) None of the other options is correct.
- (d) It focuses on training models to explicitly output the reasoning steps required for solving a task, instead of directly giving the answer.
- (e) It introduces the ability to train models with fewer data points by leveraging pre-trained embeddings.
- (f) It allows models to perform tasks by conditioning on examples provided in the input, without requiring task-specific fine-tuning.
- (g) It enables models to store task-specific knowledge directly in their architecture, removing the need for example inputs.

Question 3

[2.0 Points | No Penalty]

Consider the following Python function:

```
def calculate_total_cost(items, discount, tax_rate, coupon_code,
    membership_status):
    total_cost = sum(items)
    discounted_cost = total_cost * (1 - discount / 100)
    if coupon_code == "SUMMER21":
        discounted_cost *= 0.8 # Apply an additional 20% discount for
the coupon
    final_cost_with_tax = discounted_cost * (1 + tax_rate / 100)
    if membership_status:
        final_cost_with_tax *= 0.9 # Premium members get an additional
10% off
    if final_cost_with_tax < 0:
```

```
    final_cost_with_tax = 0
    return final_cost_with_tax
```

An LLM engine generates the following test cases:

T1: `assert calculate_total_cost([100, 50], 10, 5, "SUMMER21", True) == 102.06`

T2: `assert calculate_total_cost([200, 100], 20, 10, "WINTER21", False) == 264`

T3: `assert calculate_total_cost([50], 0, 0, "", False) == 50.0`

T4: `assert calculate_total_cost([120, 80], 15, 5, "SUMMER21", False) == 162.8`

a) What is the pass rate of the test suite against the correct code?

b) Considering the following mutations:

- **Line 3 Mutant:** `discounted_cost = total_cost * (1 - discount / 10)`
- **Line 6 Mutant:** `final_cost_with_tax = discounted_cost * (1 - tax_rate / 100)`
- **Line 9 Mutant:** `if final_cost_with_tax == 0:`

What is the mutation score of the test suite?

[Note: discuss and justify the results if needed]

Question 4

[1.0 Point | -25% Penalty]

Adapter layers are introduced to reduce the number of tunable parameters. They consist in:

- (a) A task-specific head added to the model, which is fine-tuned while the rest of the model is kept frozen.
- (b) A method for splitting a large model into smaller sub-models that are trained independently and later combined.
- (c) Replacing the feedforward layers in Transformers with low-rank matrix decompositions to reduce memory usage.
- (d) Small, trainable layers inserted into a pre-trained model, which are fine-tuned while the rest of the model is frozen.
- (e) A technique to merge multiple pre-trained models into a single model with fewer parameters.
- (f) Additional layers that replace the attention mechanism in the Transformer architecture to simplify computation.
- (g) None of the others is a valid description of adapters.

Question 5

[1.0 Point | -25% Penalty]

Which of the following is/are **not** a technique to evaluate the quality of test cases generated with LLMs?

- (a) Execution Success Rate
- (b) Coverage
- (c) Mutation score
- (d) Functional Correctness
- (e) Precision and recall

Question 6

[2.0 Points | No Penalty]

For the following encoder-only transformer model, compute:

1. The number of parameters for the embedding layer (token + positional).
2. The number of parameters of the transformer (excluding the embedding layer, or head layer(s)).

Note: Assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

- Vocabulary size: $V = 50,265$
- Embedding / model dimension: $d = 1024$
- Positional embeddings: absolute, learned, for 514 positions
- Number of layers: $L = 24$
- Number of attention heads: $H = 16$
- FF-NN projection dimension: $d_{ff} = 4096$
- Head dimension: $d_k = d_v = 64$

Report the results in Millions (M) of parameters, rounded to the nearest integer.

Question 7

[2.0 Points | No Penalty]

You are given the following sequence of characters:

a a a a b c d d d d c d d

Initially, each character is a separate token.

Apply Byte-Pair Encoding (BPE) to produce 3 additional tokens. What are the generated tokens? Write each token, in the correct order, in the following slots. Separate the merged tokens with a comma (and no spaces).

- 1st merge: _____
- 2nd merge: _____
- 3rd merge: _____

Question 8

[1.0 Point | -25% Penalty]

How does an adversarial agent interaction scheme work?

- (a) The agents work concurrently towards an individual goal, with the objective of maximizing their own benefit.
- (b) The agents work concurrently towards a common goal, with the objective of maximizing the overall benefit.
- (c) The agents work together towards a common goal, in an ordered sequence of subtasks.
- (d) The agents work together towards a common goal, in an unordered sequence of subtasks.
- (e) The agents work independently towards a common goal that is then compared.

Question 9

[1.0 Point | -25% Penalty]

The main goal of *instruction tuning* is:

- (a) To fine-tune language models to follow explicit instructions.
- (b) To reduce the size of the model without sacrificing performance.
- (c) To focus on improving the model's understanding of rare or unseen tokens.
- (d) None of the others is the main goal of instruction tuning.
- (e) To increase the model's tokenization speed by optimizing the vocabulary size.
- (f) To train language models to independently learn new languages from scratch.
- (g) To enhance the model's ability to generate longer sequences without truncation.

Question 10

[3.0 Points | No Penalty]

The goal-oriented requirements engineering (GORE) is a technique where requirements are constructed in three steps:

1. User (or agent)
2. High-level requirement
3. Low-level requirement

Consider the case where you receive natural language requirements in a string (called **REQS**). Describe an architecture of LLAMA agents to perform the steps of the GORE technique, and the prompt engineering techniques employed in each step.

For each agent, write the prompt including System, context, possible inputs from previous prompts, and expected output.

Notice: it is not necessary to write the python calls to the transformers or code for output parsing.

Question 11

[2.0 Points | No Penalty]

The attention for matrices Q, K, V (queries, keys, values) is computed as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where d_k is the dimensionality of the keys.

The following are the input tokens that reach an attention layer:

$$x = \begin{bmatrix} 3 & 4 & 1 \\ 0 & 4 & 0 \end{bmatrix}$$

The following are the W_q, W_k, W_v matrices for the attention layer:

$$W_q = \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad W_k = \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix}, \quad W_v = \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Compute the output of the attention operation **for the first token**. Write the answer reporting at least 4 significant figures:

- Dimension 1: _____
- Dimension 2: _____
- Dimension 3: _____

Question 12

[1.0 Point | -25% Penalty]

A software repository-based dataset is:

- (a) Any dataset that is based on data extracted from version control systems.
- (b) Any dataset that is composed by snippets of code.
- (c) Any dataset that is composed by fully working software code artefacts.
- (d) Any dataset that is based on open-source code available on online repositories.
- (e) Any dataset that is composed by production code artefacts paired with additional information.

2 Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1

Correct Answer: (b) and (c).

- (b) *It is possible to reduce the amount of false information provided to the final user.*
- (c) *It is possible to prevent the obsolescence of answers by using up-to-date documents.*

In-Depth Theoretical Justification:

Retrieval-Augmented Generation (RAG) integrates an information retrieval component (e.g., dense vector index using bi-encoders, BM25, or hybrid search) with a generative foundation model (Lewis et al., 2020).

- 1. Grounding and Reducing False Information (Option b):** Standard LLMs encode knowledge implicitly across parametric weights (θ). When asked about domain-specific, proprietary, or long-tail knowledge, standard auto-regressive decoding often suffers from hallucination—generating syntactically fluent but factually untrue assertions. By prepending retrieved reference passages into the context window ($\text{Context} \oplus \text{Query}$), the model conditions its attention heads directly on external factual evidence, grounding the generation and significantly curbing factual errors.
- 2. Mitigating Temporal Obsolescence (Option c):** A fundamental limitation of pre-trained LLMs is the *knowledge cutoff* date. Retraining or fine-tuning models continuously to absorb new knowledge is computationally prohibitive and susceptible to *catastrophic forgetting*. RAG decouples knowledge storage from the model parameters: updating external knowledge only requires modifying or expanding the document corpus/vector database, thereby providing the model with real-time, up-to-date information without any gradient updates.

Analysis of Distractors:

- (a) *Self-critique mechanisms:* Self-reflection or critique loops (e.g., Reflexion, Self-Refine, CRITIC) are agentic reasoning patterns that iteratively critique and revise draft outputs using feedback rules. While RAG can be combined with agentic critique, self-critique is an independent prompt engineering pattern, not a defining benefit of RAG itself.
- (d) *Fixed structure for answers:* Imposing a fixed syntactic schema (e.g., JSON mode, Pydantic validation, Outlines, Guidance) is achieved via structured prompting or logit masking during decoding, not via retrieval.
- (e) *Task decomposition:* Breaking complex problems into manageable subtasks is the hallmark of Chain-of-Thought (CoT), Least-to-Most, or Plan-and-Solve prompting, not RAG.

Solution to Question 2

Correct Answer: (f) *It allows models to perform tasks by conditioning on examples provided in the input, without requiring task-specific fine-tuning.*

In-Depth Theoretical Justification:

Prior to the emergence of Large Language Models (specifically GPT-3, Brown et al., 2020),

the dominant paradigm in Natural Language Processing was *Pre-train and Fine-tune* (e.g., BERT, RoBERTa, T5). In that regime, adapting a model to a target downstream task (such as sentiment analysis, relation extraction, or defect prediction) strictly required gradient-based optimization ($\nabla_{\theta}\mathcal{L} \neq 0$) over a supervised dataset, modifying weights and storing a dedicated model checkpoint per task.

In-Context Learning (ICL) established a radical paradigm shift:

- The model weights θ remain strictly frozen ($\nabla_{\theta} = 0$).
- Task adaptation occurs entirely at inference time through forward-pass activation conditioning.
- By feeding a natural language task description alongside a few demonstration input-output pairs $\mathcal{D}_{\text{demo}} = \{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)\}$ followed by the query x_{test} , the self-attention mechanism performs meta-gradients / implicit parameter optimization across the demonstration representations, producing the target prediction y_{test} without altering a single weight.

Analysis of Distractors:

- (a) *Replaces attention by sequential processing*: Factually incorrect. ICL relies heavily on multi-head attention to attend across context demonstrations; sequential processing refers to recurrent architectures (RNNs/LSTMs).
- (b) *Compressing longer sequences into fixed-size representations*: This refers to sentence-level pooling or bottleneck autoencoders, which is unrelated to ICL.
- (c) *None of the other options is correct*: Incorrect because option (f) is the exact canonical definition of ICL.
- (d) *Explicitly output reasoning steps*: This defines Chain-of-Thought (CoT) prompting (Wei et al., 2022), which is a specialized variant of prompting, whereas ICL is the broader paradigm encompassing zero-shot and few-shot conditioning.
- (e) *Fewer data points leveraging pre-trained embeddings*: This describes traditional transfer learning / linear probing on frozen embeddings.
- (g) *Store task-specific knowledge directly in architecture*: This describes parametric fine-tuning, the exact opposite of ICL's non-parametric context conditioning.

Solution to Question 3

Final Answers:

- **a) Pass Rate: 75.0%** (3 out of 4 tests pass).
- **b) Mutation Score: 66.7%** (2 out of 3 mutants killed).

Step-by-Step Derivation and Analysis:

Part a) Evaluation of Test Suite on Correct Code

Let us trace the execution of the function `calculate_total_cost` for each test case:

1. **Test T1:** `calculate_total_cost([100, 50], 10, 5, "SUMMER21", True)`

- $\text{total_cost} = 100 + 50 = 150.0$
- $\text{discounted_cost} = 150.0 \times (1 - 10/100) = 150.0 \times 0.90 = 135.0$
- Coupon "SUMMER21": $\text{discounted_cost} = 135.0 \times 0.80 = 108.0$
- Tax (5%): $\text{final_cost_with_tax} = 108.0 \times (1 + 5/100) = 108.0 \times 1.05 = 113.4$
- Member discount (10%): $\text{final_cost_with_tax} = 113.4 \times 0.90 = 102.06$
- Negative check: $102.06 \not\leq 0$
- Returned: 102.06. Assertion: `== 102.06` → **PASS**.

2. Test T2: `calculate_total_cost([200, 100], 20, 10, "WINTER21", False)`

- $\text{total_cost} = 200 + 100 = 300.0$
- $\text{discounted_cost} = 300.0 \times (1 - 20/100) = 300.0 \times 0.80 = 240.0$
- Coupon "WINTER21" $\neq$ "SUMMER21": no extra discount → 240.0
- Tax (10%): $\text{final_cost_with_tax} = 240.0 \times (1 + 10/100) = 240.0 \times 1.10 = 264.0$
- Member discount: `membership_status` is `False` → 264.0
- Returned: 264.0. Assertion: `== 264` → **PASS**.

3. Test T3: `calculate_total_cost([50], 0, 0, "", False)`

- $\text{total_cost} = 50.0$
- $\text{discounted_cost} = 50.0 \times (1 - 0) = 50.0$
- Coupon check fails → 50.0
- Tax (0%): $\text{final_cost_with_tax} = 50.0 \times 1.0 = 50.0$
- Member discount: `False` → 50.0
- Returned: 50.0. Assertion: `== 50.0` → **PASS**.

4. Test T4: `calculate_total_cost([120, 80], 15, 5, "SUMMER21", False)`

- $\text{total_cost} = 120 + 80 = 200.0$
- $\text{discounted_cost} = 200.0 \times (1 - 15/100) = 200.0 \times 0.85 = 170.0$
- Coupon "SUMMER21": $\text{discounted_cost} = 170.0 \times 0.80 = 136.0$
- Tax (5%): $\text{final_cost_with_tax} = 136.0 \times (1 + 5/100) = 136.0 \times 1.05 = 142.8$
- Member discount: `False` → 142.8
- Returned: **142.8**. Assertion expects: `== 162.8`.
- Since $142.8 \neq 162.8$, the assertion raises an `AssertionError` → **FAIL**.

Thus, 3 out of 4 tests pass:

$$\text{Pass Rate} = \frac{3}{4} = 75.0\%$$

Part b) Mutation Score Analysis

A mutant is considered *killed* if at least one passing test case in the suite produces an unexpected result (assertion failure or error) when executed against the mutant code.

1. Mutant 1 (Line 3): `discounted_cost = total_cost * (1 - discount / 10)`

- Under T1: `discount = 10` $\implies 1 - 10/10 = 0.0$. The function evaluates to 0.0.
- Since the expected value is 102.06, the assertion fails ($0.0 \neq 102.06$).

- **Mutant 1 is KILLED by T1** (as well as T2, where $1 - 20/10 = -1.0$).
- 2. Mutant 2 (Line 6):** `final_cost_with_tax = discounted_cost * (1 - tax_rate / 100)`
 - Under T1: Tax subtraction yields $108.0 \times (1 - 0.05) \times 0.9 = 108.0 \times 0.95 \times 0.9 = 92.34$.
 - Since $92.34 \neq 102.06$, the assertion fails.
 - **Mutant 2 is KILLED by T1** (as well as T2, where $240 \times 0.9 = 216.0 \neq 264$).
- 3. Mutant 3 (Line 9):** `if final_cost_with_tax == 0:` (replacing `< 0`)
 - In the original code, if `final_cost_with_tax < 0`, it is clamped to 0.
 - In the mutant, if `final_cost_with_tax == 0`, it is set to 0.
 - For all non-negative numbers (≥ 0), the original code leaves the value unchanged. The mutant also leaves strictly positive values unchanged, and reassigns 0 to 0.
 - For all test cases in the test suite (T1, T2, T3, and even T4), the calculated cost is strictly positive ($102.06, 264, 50.0, 142.8 > 0$). Neither the original `< 0` branch nor the mutant `== 0` branch is triggered to alter the return value.
 - Consequently, Mutant 3 produces the exact same output as the original code on all test cases. No test detects the fault.
 - **Mutant 3 SURVIVES (is NOT killed).**

Out of 3 mutants, 2 are killed:

$$\text{Mutation Score} = \frac{\text{Mutants Killed}}{\text{Total Mutants}} = \frac{2}{3} \approx 66.7\%$$

Solution to Question 4

Correct Answer: (d) *Small, trainable layers inserted into a pre-trained model, which are fine-tuned while the rest of the model is frozen.*

In-Depth Theoretical Justification:

In Parameter-Efficient Fine-Tuning (PEFT), **Adapter Layers** (pioneered by Houlsby et al., 2019, and Pfeiffer et al., 2020) were designed as an alternative to full fine-tuning.

- **Architecture:** An adapter is a small, bottleneck feed-forward network inserted internally between the standard sub-layers of each Transformer block (typically after the multi-head self-attention and after the position-wise feed-forward network). It consists of a down-projection matrix $W_{\text{down}} \in \mathbb{R}^{d \times m}$ (where intermediate bottleneck dimension $m \ll d$), a non-linear activation $\sigma(\cdot)$, and an up-projection matrix $W_{\text{up}} \in \mathbb{R}^{m \times d}$, accompanied by a residual connection:

$$h_{\text{out}} = h + \sigma(hW_{\text{down}})W_{\text{up}}$$

- **Training Dynamics:** During task-specific fine-tuning, all pre-trained Transformer weights θ_{base} are frozen ($\nabla_{\theta_{\text{base}}} = 0$). Only the newly introduced adapter weights ($\approx 0.5\% - 3\%$ of total model size) are trained. This vastly reduces storage requirements and eliminates catastrophic forgetting across tasks.

Analysis of Distractors:

- (a) *Task-specific head added to the model*: This refers to linear probing / classification head fine-tuning, which only updates the top projection layer without modifying intermediate hidden state representations.
- (b) *Splitting a model into smaller sub-models*: This describes model parallelism, pipeline parallelism, or Mixture of Experts (MoE) routing.
- (c) *Replacing feedforward layers with low-rank decompositions*: This describes Low-Rank Adaptation (LoRA, Hu et al., 2021) or weight factorization methods, which reparametrize weight updates rather than adding distinct architectural layers.
- (e) *Merging multiple models*: This describes model merging algorithms (e.g., Task Arithmetic, TIES-Merging, SLERP).
- (f) *Replacing attention mechanisms*: This describes linear attention approximations or state-space models (e.g., Mamba, RWKV).
- (g) *None of the others*: Incorrect because (d) is the exact canonical definition.

Solution to Question 5

Correct Answer: (d) *Functional Correctness*.

In-Depth Theoretical Justification:

In software engineering literature concerning LLM-based code and test generation (e.g., Chen et al., 2021 [HumanEval]; Lemieux et al., 2023 [CODATMOS]; Schäfer et al., 2023 [TestPilot]):

- **Functional Correctness** (measured via $\text{pass}@k$) is the gold standard benchmark metric used to evaluate **code generation** (synthesis of implementation code given a natural language prompt or function signature). In this setting, the generated code's functional correctness is evaluated by executing it against a pre-existing ground truth unit test suite.
- When the task is inverted—evaluating the **quality of generated test cases** themselves—Functional Correctness is not an applicable metric because test cases act as the *evaluators* (the specification oracle), not the target under test. Running a generated test against an implementation cannot determine test correctness: a failing test could indicate either an effective bug-revealing test or a broken, hallucinated test oracle.
- Quality of generated test suites must therefore be measured via dedicated test assessment criteria:
 1. **Execution Success Rate (ESR)**: Percentage of generated tests that compile and execute without unhandled runtime exceptions or environment errors.
 2. **Code Coverage**: Degree to which the test suite covers source lines, branches, and execution paths of the focal method.
 3. **Mutation Score**: Proportion of artificial faults (mutants) successfully detected and killed by the generated assertions.
 4. **Precision and Recall**: Quantitative measurement of whether assertions correctly distinguish valid system states from true defects (avoiding false alarms due to hallucinated assertions).

Solution to Question 6

Final Answers:

- **Embedding Size:** 52 M parameters.
- **Transformer Size:** 302 M parameters.

Step-by-Step Mathematical Derivations:

The problem specifies an encoder-only Transformer architecture (identical to **RoBERTa-large**) under the following parameters:

- Vocabulary size $V = 50,265$
- Model / Hidden dimension $d = 1024$
- Positional embeddings: absolute, learned, for $N_{\text{pos}} = 514$ positions
- Number of Transformer layers $L = 24$
- Attention heads $H = 16$, with head dimension $d_k = d_v = 64$ ($H \times d_k = 16 \times 64 = 1024 = d$)
- Feed-Forward Network (FFN) intermediate dimension $d_{ff} = 4096$
- Constraints: No bias terms, ignore layer normalization parameters.

1. Embedding Layer Parameters

The embedding layer consists of the token embedding lookup table and the absolute learned position embedding lookup table:

$$P_{\text{token}} = V \times d = 50,265 \times 1024 = 51,471,360$$

$$P_{\text{pos}} = N_{\text{pos}} \times d = 514 \times 1024 = 526,336$$

Summing both components:

$$P_{\text{embed}} = P_{\text{token}} + P_{\text{pos}} = 51,471,360 + 526,336 = 51,997,696 \text{ parameters}$$

Expressed in Millions (M) and rounded to the nearest integer:

$$\frac{51,997,696}{10^6} \approx 51.9977 \text{ M} \xrightarrow{\text{round}} \mathbf{52 \text{ M}}$$

2. Transformer Layers Parameters (Excluding Embeddings and Heads)

Each of the $L = 24$ encoder layers comprises two primary sub-blocks: Multi-Head Self-Attention (MHA) and the Position-wise Feed-Forward Network (FFN).

a) Multi-Head Attention (MHA): There are four weight matrices per layer: Queries (W_Q), Keys (W_K), Values (W_V), and Output Projection (W_O). With no biases:

$$W_Q \in \mathbb{R}^{d \times d} \implies 1024 \times 1024 = 1,048,576$$

$$W_K \in \mathbb{R}^{d \times d} \implies 1024 \times 1024 = 1,048,576$$

$$W_V \in \mathbb{R}^{d \times d} \implies 1024 \times 1024 = 1,048,576$$

$$W_O \in \mathbb{R}^{d \times d} \implies 1024 \times 1024 = 1,048,576$$

$$P_{\text{MHA}} = 4 \times d^2 = 4 \times 1,048,576 = 4,194,304 \text{ parameters/layer}$$

b) Feed-Forward Network (FFN): The two-layer MLP projects from $d \rightarrow d_{ff}$ and back from $d_{ff} \rightarrow d$:

$$W_1 \in \mathbb{R}^{d \times d_{ff}} \implies 1024 \times 4096 = 4,194,304$$

$$W_2 \in \mathbb{R}^{d_{ff} \times d} \implies 4096 \times 1024 = 4,194,304$$

$$P_{\text{FFN}} = 2 \times d \times d_{ff} = 2 \times 4,194,304 = 8,388,608 \text{ parameters/layer}$$

c) Total Transformer Parameters: Total parameters per layer:

$$P_{\text{layer}} = P_{\text{MHA}} + P_{\text{FFN}} = 4,194,304 + 8,388,608 = 12,582,912 \text{ parameters}$$

Across all $L = 24$ layers:

$$P_{\text{transformer}} = 24 \times P_{\text{layer}} = 24 \times 12,582,912 = 301,989,888 \text{ parameters}$$

Expressed in Millions (M) and rounded to the nearest integer:

$$\frac{301,989,888}{10^6} \approx 301.9899 \text{ M} \xrightarrow{\text{round}} \mathbf{302 \text{ M}}$$

Note on Official Model Checkpoint: In the full RoBERTa-large implementation, including token type embeddings ($1 \times 1024 = 1024$), LayerNorm parameters ($2 \times 1024 = 2048$ in embeddings, $4 \times 1024 = 4096$ per layer), and bias vectors ($4 \times 1024 + 4096 + 1024 = 9216$ per layer), the exact counts are 52,000,768 (Embedding) and 302,309,376 (Transformer), which also round identically to 52 M and 302 M.

Solution to Question 7

Correct Ordered Merges:

- 1st merge: d,d
- 2nd merge: a,a
- 3rd merge: c,dd

Step-by-Step BPE Derivation:

We are given the initial sequence of individual character tokens:

[a, a, a, a, b, c, d, d, d, d, c, d, d]

Total initial length = 13 tokens.

Iteration 1:

Count the frequency of all adjacent token pairs:

Pair	Occurrences / Positions	Frequency
(a, a)	(0, 1), (1, 2), (2, 3)	3
(a, b)	(3, 4)	1
(b, c)	(4, 5)	1
(c, d)	(5, 6), (10, 11)	2
(d, d)	(6, 7), (7, 8), (8, 9), (11, 12)	4

The most frequent pair is (d, d) with frequency 4. Merge d,d into a new token dd. Applying greedy, left-to-right, non-overlapping replacement:

- d d d d at positions 6–9 becomes dd dd
- d d at positions 11–12 becomes dd

⇒ **1st merge: d,d**

Iteration 2:

Updated token sequence:

[a, a, a, a, b, c, dd, dd, c, dd]

Count the adjacent pairs:

Pair	Occurrences / Positions	Frequency
(a, a)	(0, 1), (1, 2), (2, 3)	3
(a, b)	(3, 4)	1
(b, c)	(4, 5)	1
(c, dd)	(5, 6), (8, 9)	2
(dd, dd)	(6, 7)	1
(dd, c)	(7, 8)	1

The most frequent pair is (a, a) with frequency 3. Merge a,a into a new token aa. Applying greedy, left-to-right, non-overlapping replacement:

- a a a a becomes aa aa

⇒ **2nd merge: a,a**

Iteration 3:

Updated token sequence:

[aa, aa, b, c, dd, dd, c, dd]

Count the adjacent pairs:

Pair	Occurrences / Positions	Frequency
(aa, aa)	(0, 1)	1
(aa, b)	(1, 2)	1
(b, c)	(2, 3)	1
(c, dd)	(3, 4), (6, 7)	2
(dd, dd)	(4, 5)	1
(dd, c)	(5, 6)	1

The most frequent pair is (c, dd) with frequency 2. Merge c,dd into a new token cdd.

⇒ **3rd merge: c,dd**

Solution to Question 8

Correct Answer: (a) *The agents work concurrently towards an individual goal, with the objective of maximizing their own benefit.*

In-Depth Theoretical Justification:

In the taxonomy of Multi-Agent Systems (MAS) and LLM-based collaborative architectures (e.g., Wang et al., 2023; Du et al., 2023 [ChatEval]; Liang et al., 2023):

- **Adversarial Interaction (Game-Theoretic / Competitive):** Agents operate under non-cooperative game theory (such as zero-sum games or multi-agent debate). Each agent possesses an autonomous, individual utility function and strives concurrently or iteratively to maximize its own objective or benefit. Examples include:
 - *Red-Teaming vs. Blue-Teaming:* An attacking agent generates security exploits or jailbreaks, while a defending agent generates patches or guards.
 - *Debate Frameworks:* Two agents take opposing stances on code design or bug identification to stress-test logical consistency, where each agent’s objective is to disprove the opponent’s arguments.
- **Collaborative / Cooperative Interaction:** Agents share a unified global utility function and cooperate to maximize collective welfare (represented by options b, c, and d).

Analysis of Distractors:

- (b) *Concurrently towards a common goal:* This defines **concurrent collaborative** interaction (e.g., swarm intelligence, independent parallel sub-task workers).
- (c) *Together towards a common goal in an ordered sequence:* This defines a **sequential pipeline** collaborative architecture (e.g., Waterfall agent chains: Specifier → Coder → Tester).
- (d) *Together towards a common goal in an unordered sequence:* This defines a **black-board / reactive** cooperative architecture.
- (e) *Work independently towards a common goal that is then compared:* This defines **multi-agent ensembling / consensus voting** (e.g., Self-Consistency with majority voting), where all agents share the same target goal.

Solution to Question 9

Correct Answer: (a) *To fine-tune language models to follow explicit instructions.*

In-Depth Theoretical Justification:

Foundation language models trained via self-supervised pre-training optimize next-token prediction over web text:

$$\mathcal{L}_{\text{pretrain}}(\theta) = - \sum_{t=1}^T \log P(x_t \mid x_{<t}; \theta)$$

While this imparts vast linguistic and world knowledge, raw base models act merely as *document completers*. When presented with an instruction (e.g., "Write a Python function to compute Fibonacci numbers"), a pre-trained base model may hallucinate additional interview questions, repeat the prompt, or append irrelevant blog text rather than fulfilling the command.

Instruction Tuning (e.g., FLAN [Wei et al., 2021], T0 [Sanh et al., 2021], InstructGPT

[Ouyang et al., 2022], Alpaca [Taori et al., 2023]) bridges this alignment gap:

- The model is supervised on diverse datasets formatted as instruction-input-response tuples: $(\mathcal{I}, x, y)$.
- The objective is to teach the model to interpret the semantic intent of the instruction $\mathcal{I}$ and generate the compliant, helpful response y .
- Crucially, instruction tuning enables strong zero-shot generalization to entirely unseen tasks specified via natural language instructions.

Analysis of Distractors:

- (b) *Reduce model size*: This is the goal of model compression, pruning, quantization, or knowledge distillation (e.g., DistilBERT).
- (c) *Improve understanding of rare tokens*: This pertains to subword vocabulary optimization (byte fallback BPE, WordPiece).
- (d) *None of the others*: Incorrect because (a) is the definitive objective.
- (e) *Increase tokenization speed*: This relates to vocabulary size engineering and C++/Rust tokenizer implementations (e.g., Hugging Face Tokenizers).
- (f) *Learn new languages from scratch*: This refers to multilingual pre-training or vocabulary transfer.
- (g) *Generate longer sequences without truncation*: This refers to context window extension techniques (e.g., RoPE scaling, YaRN, ALiBi).

Solution to Question 10

Goal-Oriented Requirements Engineering (GORE) Multi-Agent Architecture

1. Architecture Description

Goal-Oriented Requirements Engineering (GORE) models requirements by decomposing high-level stakeholder aspirations into concrete, operationalized software specifications through three hierarchical abstractions:

Actors / Users $\longrightarrow$ High-Level Requirements (Goals) $\longrightarrow$ Low-Level Requirements (Specifications)

To automate GORE from an unstructured requirements string (REQS), we employ a **Sequential Multi-Agent Pipeline** with intermediate state propagation:

- **Agent 1 (Actor Extraction Agent)**: Identifies all human and external system actors that interact with the system.
- **Agent 2 (High-Level Goal Agent)**: Takes REQS and the extracted actors to synthesize strategic, functional, and quality goals (the “why” and “what”).
- **Agent 3 (Low-Level Operationalization Agent)**: Takes REQS, the actors, and high-level goals to derive technical specifications, concrete data models, class contracts, and interface requirements (the “how”).

2. Prompt Engineering Techniques Employed

1. **Role / Persona Prompting:** Each agent is assigned an expert persona (System Analyst, Product Manager, Software Architect) to steer stylistic and technical output.
2. **Few-Shot In-Context Learning (ICL):** High-quality demonstration exemplars with diverse domains (e.g., e-commerce, physical library systems) are provided to establish strict output schemas and prevent prose drift.
3. **Structured Output Formatting:** Output is constrained to JSON / Markdown formats with explicit key structures to enable reliable parsing and chaining between agents.
4. **Context Chaining:** Upstream outputs are injected as structured context variables into downstream agent prompts to maintain semantic consistency across the pipeline.

3. Complete Agent Specifications and Prompts

Agent 1: Actor & User Extraction Agent

- **System Role:** Expert Requirements Analyst specializing in stakeholder and actor identification.
- **Input:** {REQS}
- **Output Schema:** A structured list of distinct users/actors with role descriptions.

SYSTEM:

You are an expert System Analyst specializing in Goal-Oriented Requirements Engineering (GORE). Your objective is to analyze natural language requirements descriptions and extract all primary users, secondary actors, and external systems that interact with the target software.

FEW-SHOT EXAMPLES:

Example 1:

Input REQS: "A physical library management system where members can browse books, check them out, and return them. Librarians manage book inventories, register new patrons, and contact suppliers for restocks."

Output:

- Member / Patron: End user who browses, borrows, and returns library books.
- Librarian: Administrative user who manages book inventory and registers patrons.
- Book Supplier: External partner contacted for acquisition and catalog restocking.

Example 2:

Input REQS: "An online e-commerce platform allowing customers to purchase electronic devices, track order deliveries, and process payments via Stripe. Warehouse staff packages items and initiates shipping."

Output:

- Customer: End user purchasing electronics and tracking shipments.
- Warehouse Staff: Operational user packaging orders and coordinating logistics.
- Stripe Payment Gateway: External third-party system processing financial transactions.

```
CURRENT TASK:
Extract all users and actors from the following requirements
  description:
### Input REQS:
{REQS}

### Output:
```

Agent 2: High-Level Goal & Functional Requirement Agent

- **System Role:** Senior Product Manager formulating functional objectives and stakeholder goals.
- **Input:** {REQS}, {EXTRACTED_ACTORS}
- **Output Schema:** Structured list mapping each actor to their respective high-level strategic goals.

```
SYSTEM:
You are a Senior Product Manager specializing in Goal-Oriented
Requirements Engineering. Your task is to formulate high-level
functional goals (the "what" and "why") for each identified actor,
using the raw requirements text and the list of actors.

FEW-SHOT EXAMPLES:
### Example 1:
Input REQS: "A physical library system where members borrow books and
  librarians manage stock."
Actors: Member, Librarian
Output:
- Goal 1 (Member): Enable patrons to search available books and borrow
  them for a fixed loan duration.
- Goal 2 (Member): Enable patrons to return borrowed items and review
  active loans.
- Goal 3 (Librarian): Maintain an accurate inventory of physical book
  copies and catalog new arrivals.

### Example 2:
Input REQS: "An online electronics shop with customer ordering and
  warehouse packaging."
Actors: Customer, Warehouse Staff
Output:
- Goal 1 (Customer): Provide an intuitive product search, shopping cart
  , and seamless checkout workflow.
- Goal 2 (Customer): Deliver real-time tracking updates regarding
  package shipping status.
- Goal 3 (Warehouse Staff): Efficiently process incoming order queues
  and dispatch fulfillment notices.

CURRENT TASK:
Formulate high-level requirements based on the requirements and
  extracted actors below:
### Input REQS:
{REQS}
### Extracted Actors:
{EXTRACTED_ACTORS}
```

```
### Output:
```

Agent 3: Low-Level Operationalization & Specification Agent

- **System Role:** Lead Software Architect defining technical specifications, data contracts, and class designs.
- **Input:** {REQS}, {EXTRACTED_ACTORS}, {HIGH_LEVEL_GOALS}
- **Output Schema:** Technical requirements, domain class models, and invariant specifications.

```
SYSTEM:
```

```
You are a Lead Software Architect. Your task is to operationalize high-level goals into concrete, low-level technical specifications. This includes specifying domain classes, methods, data schemas, and functional validation constraints.
```

```
FEW-SHOT EXAMPLES:
```

```
### Example 1:
```

```
Input Goals: "Enable patrons to search available books and borrow them; Maintain book inventory."
```

```
Output:
```

- Spec 1: Implement class 'Book' with attributes 'isbn: str', 'title: str', 'status: Enum('AVAILABLE', 'LOANED')'.
- Spec 2: Implement 'LoanService.borrow_book(patron_id: UUID, book_id: UUID) -> LoanReceipt' enforcing invariant that overdue accounts cannot borrow.
- Spec 3: Store loan records in relational table 'loans' with foreign keys to 'patron' and 'book'.

```
### Example 2:
```

```
Input Goals: "Provide product search and checkout; Deliver shipment tracking."
```

```
Output:
```

- Spec 1: Implement class 'Order' with 'order_id: UUID', 'items: List[OrderItem]', 'total: Decimal', 'state: OrderState'.
- Spec 2: Implement REST endpoint 'POST /api/v1/checkout' accepting validated billing addresses and payment tokens.
- Spec 3: Emit event 'OrderDispatchedEvent' containing carrier tracking number to notify notification workers.

```
CURRENT TASK:
```

```
Derive low-level technical requirements based on the context provided:
```

```
### Input REQS:
```

```
{REQS}
```

```
### Extracted Actors:
```

```
{EXTRACTED_ACTORS}
```

```
### High-Level Goals:
```

```
{HIGH_LEVEL_GOALS}
```

```
### Output:
```

Solution to Question 11**Correct Output for Token 1 (Reported to at least 4 significant figures):**

- **Dimension 1: 3.1956** (Exact: 3.195570314)
- **Dimension 2: 1.6089** (Exact: 1.608859364)
- **Dimension 3: -5.6310** (Exact: -5.631007774)

Complete Step-by-Step Mathematical Derivation:

We are given the scaled dot-product attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The input token matrix containing two sequence tokens (x_1 and x_2) is:

$$x = \begin{bmatrix} 3 & 4 & 1 \\ 0 & 4 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 3}, \quad \text{where } x_1 = [3, 4, 1] \text{ and } x_2 = [0, 4, 0]$$

The projection weight matrices are:

$$W_q = \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad W_k = \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad W_v = \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \in \mathbb{R}^{3 \times 3}$$

Notice that W_q and W_k project into dimension $d_k = 2$. Therefore, the scaling factor is:

$$\sqrt{d_k} = \sqrt{2} \approx 1.41421356$$

Step 1: Compute Linear Projections Q , K , and V **Query Matrix $Q = xW_q$:**

$$\begin{aligned} q_1 &= x_1 W_q = [3, 4, 1] \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix} \\ &= [3(-2) + 4(1) + 1(0), \quad 3(-1) + 4(1) + 1(-1)] \\ &= [-6 + 4 + 0, \quad -3 + 4 - 1] = [-2, \quad 0] \\ q_2 &= x_2 W_q = [0, 4, 0] \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix} \\ &= [0(-2) + 4(1) + 0(0), \quad 0(-1) + 4(1) + 0(-1)] = [4, \quad 4] \\ Q &= \begin{bmatrix} -2 & 0 \\ 4 & 4 \end{bmatrix} \end{aligned}$$

Key Matrix $K = xW_k$:

$$\begin{aligned}
 k_1 &= x_1 W_k = [3, 4, 1] \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix} \\
 &= [3(0) + 4(-1) + 1(-1), \quad 3(-2) + 4(0) + 1(1)] \\
 &= [0 - 4 - 1, \quad -6 + 0 + 1] = [-5, \quad -5] \\
 k_2 &= x_2 W_k = [0, 4, 0] \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix} \\
 &= [0(0) + 4(-1) + 0(-1), \quad 0(-2) + 4(0) + 0(1)] = [-4, \quad 0] \\
 K &= \begin{bmatrix} -5 & -5 \\ -4 & 0 \end{bmatrix}
 \end{aligned}$$

Value Matrix $V = xW_v$:

$$\begin{aligned}
 v_1 &= x_1 W_v = [3, 4, 1] \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \\
 &= [3(0) + 4(1) + 1(-1), \quad 3(1) + 4(0) + 1(-1), \quad 3(-2) + 4(0) + 1(-1)] \\
 &= [0 + 4 - 1, \quad 3 + 0 - 1, \quad -6 + 0 - 1] = [3, \quad 2, \quad -7] \\
 v_2 &= x_2 W_v = [0, 4, 0] \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \\
 &= [0(0) + 4(1) + 0(-1), \quad 0(1) + 4(0) + 0(-1), \quad 0(-2) + 4(0) + 0(-1)] = [4, \quad 0, \quad 0] \\
 V &= \begin{bmatrix} 3 & 2 & -7 \\ 4 & 0 & 0 \end{bmatrix}
 \end{aligned}$$

Step 2: Compute Raw Attention Scores for Token 1

The attention score for token 1 attending to token j is given by:

$$S_{1,j} = \frac{q_1 \cdot k_j}{\sqrt{d_k}}$$

- Dot product with $k_1 = [-5, -5]$:

$$q_1 \cdot k_1 = (-2)(-5) + (0)(-5) = 10 + 0 = 10$$

Scaled score:

$$S_{1,1} = \frac{10}{\sqrt{2}} = 5\sqrt{2} \approx 7.071067812$$

- Dot product with $k_2 = [-4, 0]$:

$$q_1 \cdot k_2 = (-2)(-4) + (0)(0) = 8 + 0 = 8$$

Scaled score:

$$S_{1,2} = \frac{8}{\sqrt{2}} = 4\sqrt{2} \approx 5.656854249$$

Step 3: Apply Softmax to Obtain Attention Weights

The attention weights for the first token are:

$$A_{1,1} = \frac{e^{S_{1,1}}}{e^{S_{1,1}} + e^{S_{1,2}}}, \quad A_{1,2} = \frac{e^{S_{1,2}}}{e^{S_{1,1}} + e^{S_{1,2}}}$$

Using numerically stable softmax:

$$S_{1,1} - S_{1,2} = 5\sqrt{2} - 4\sqrt{2} = \sqrt{2} \approx 1.41421356$$

$$A_{1,1} = \frac{1}{1 + e^{-\sqrt{2}}} = \frac{1}{1 + 0.24311673} \approx \mathbf{0.804429683}$$

$$A_{1,2} = \frac{e^{-\sqrt{2}}}{1 + e^{-\sqrt{2}}} = 1 - A_{1,1} \approx \mathbf{0.195570317}$$

Step 4: Compute Output Representation for Token 1

The final representation is the weighted combination of the value vectors v_1 and v_2 :

$$\text{Output}_1 = A_{1,1}v_1 + A_{1,2}v_2$$

Substituting $v_1 = [3, 2, -7]$ and $v_2 = [4, 0, 0]$:

$$\begin{aligned} \text{Dimension 1} &= 0.804429683 \times 3 + 0.195570317 \times 4 \\ &= 2.413289049 + 0.782281268 = \mathbf{3.195570317} \approx \mathbf{3.1956} \end{aligned}$$

$$\begin{aligned} \text{Dimension 2} &= 0.804429683 \times 2 + 0.195570317 \times 0 \\ &= 1.608859366 + 0.0 = \mathbf{1.608859366} \approx \mathbf{1.6089} \end{aligned}$$

$$\begin{aligned} \text{Dimension 3} &= 0.804429683 \times (-7) + 0.195570317 \times 0 \\ &= -5.631007781 + 0.0 = \mathbf{-5.631007781} \approx \mathbf{-5.6310} \end{aligned}$$

These values match the examination answer keys to full precision.

Solution to Question 12

Correct Answer: (a) *Any dataset that is based on data extracted from version control systems.*

In-Depth Theoretical Justification:

In Mining Software Repositories (MSR) and empirical software engineering:

- A **software repository-based dataset** is defined strictly by its extraction from Version Control Systems (VCS, such as Git, Mercurial, or SVN).
- Unlike static code corpora, repository datasets capture the *temporal evolution* and *collaborative lifecycle* of software systems. This includes commit logs, diffs / patches, author attribution, commit messages, parent-child commit DAG structures, branch histories, merge conflicts, pull requests, and links to issue tracking systems.
- Benchmark examples heavily utilized in LLM4SE include:
 - *Defects4J*: Real bug fixes extracted from Git commit histories of popular Java

projects.

- *CommitBench* / *GitBug*: Datasets constructed to benchmark automated commit message generation and vulnerability patch localization.
- *ManySStub4J*: Single-statement bug fixes mined directly from Git change deltas.

Analysis of Distractors:

- (b) *Composed by snippets of code*: Snippets can be mined from competitive coding websites, textbooks, or online forums (e.g., StackOverflow) without any repository context.
- (c) *Composed by fully working software code artefacts*: Standalone executable benchmarks (e.g., HumanEval, MBPP) consist of complete functions and test cases, but lack repository evolution data.
- (d) *Based on open-source code available on online repositories*: Merely cloning files from an open-source host without mining VCS history yields static source code, not a repository-based dataset.
- (e) *Production code paired with additional information*: This describes general enriched code datasets (e.g., binaries with debugging symbols), which do not necessarily derive from version control systems.